A Lock-Assisted Lightweight Quorum Validation Model for Double-Spend Prevention in Hash-Chain IoT Micropayments

Mustafa Can Çalışkan
Computer Engineering Department
Istanbul Technical University
Istanbul, Turkey
caliskanmu20@itu.edu.tr

***Abstract*—IoT micropayments require low fees and fast validation, but blockchain-heavy and broker-based approaches are often too costly, too slow, or vulnerable to double-spending. We present a lightweight quorum validation model for hash-chain micropayments that uses a small permissioned validator set and a lock-assisted two-phase protocol. Merchants submit received tokens to validators, and payments are accepted only after quorum approval, while token locking prevents concurrent reuse of the same token. We analyze the safety and liveness conditions of the model and implement a distributed prototype to evaluate its behavior under normal operation, double-spend attempts, and validator faults. The results show that the approach prevents double-spending in safe configurations while maintaining practical performance for small-scale IoT payment settings.**

***Index Terms*—micropayment, double-spend prevention, hash chain, quorum validation, IoT, byzantine fault tolerance, lock/lease protocol**

I. Introduction

The Internet of Things (IoT) ecosystem is growing exponentially, with a multitude of new use cases emerging, most of which necessitate small and frequent payments exchanged between IoT devices, smart sensors, EV charging, APIs, etc. [1] The Internet, at present, can typically execute a fraction of the numerous daily payments needed to maintain the rapidly expanding IoT ecosystem and blockchain scalability and fees make micropayments non-practical for many of these use cases [2].

Hash-chain based micropayments are an approach for enabling micropayments over broadcast-medium network channels, where the payment originator creates a hash chain and successively publishes tokens of it, which can be redeemed in exchange for real currency. This technique usually uses a cheap one-time hash verification per micropayment, rather than being subjected to potentially high blockchain transaction fees [3]. It, however, typically relies on a central broker or intermediary for settlement, thereby failing to resolve the problem of double spending in a decentralized context.

Payment channel networks (PCNs) are a popular approach for making payments scalable, as it allows two parties to exchange thousands or millions of payments without incurring per-transaction fees or waiting for multiple confirmations [4]. Examples such as the Lightning Network permit two users to engage in many off-chain transactions [5]. However, PCNs necessitate locking up funds into the channels. Parties must be online concurrently for all of these operations and suffer from complex routing mechanisms in larger networks [6], [7]. In the context of low-powered resource-constrained devices often found in the IoT sphere, this overhead is frequently too burdensome for them to overcome.

A recently developed technique in the academic research utilizes the concept of byzantine quorum systems to support the validation of many parallel payments [8]. In these settings, transactions can be validated and trusted, although not necessarily achieving full consensus amongst every party in the network. It doesn't specifically leverage the potential value to the extent of hash-chain tokenized micropayments nor adequately manage issues of race conditions in a system where a hash chain token could possibly be transmitted to two different merchants concurrently.

To address these issues, in this article, we introduce a novel lock-assisted lightweight quorum validation model designed for hash-chain IoT micropayments. Our proposed model employs a small group of trustworthy permissioned validators along with a two-phase lock/lease mechanism that effectively stops double spending issues. The hash chain sender generates hash tokens and reveals them one by one to each of the different merchants involved. A merchant receives the token and promptly broadcasts it to all participating validators. At least a quorum q of validator approvals are required for a transaction to be recognized and accepted. The locking phase ensures that no single token is processed or approved by multiple merchants at the same time.

To the best of our knowledge, no prior published work has simultaneously integrated hash-chain tokenized micropayments along with guaranteed deterministic quorum-based validation and a two-phase locking/leasing mechanism to efficiently resolve concurrent same-token submission race problems in a small and trusted IoT ecosystem.

We make the following contributions:

1) We propose a two-phase lock/lease validation protocol

that makes the check-then-mark race condition obsolete in hash-chain micropayments, without introducing additional blockchain infrastructure.
2) We formally express the system's safety condition ($2q - n \geq f_b + 1$, where f_b bounds byzantine failures) and liveness condition ($n - f_c \geq q$, where f_c bounds crash failures), and derive the valid quorum range $q \in \left[\left\lceil \frac{n+f_b+1}{2} \right\rceil, n - f_c\right]$.
3) We implement a distributed proof-of-concept prototype and experimentally evaluate it across 14 different configurations involving byzantine faults, crash faults, and throughput under a statically configured validator set.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the proposed solution. Section IV presents the experimental evaluation. Section V concludes the paper.

II. IoT Micropayment Protocols and Double-Spend Prevention

We focus our literature review on three areas: IoT micropayment protocols, payment channel networks (PCNs), and distributed double-spend prevention methods.

A. IoT Micropayment Protocols

Micropayment protocols for IoT have to address the IoT system requirements of fast speed, cost-effectiveness and low resource usage. In [1], the authors introduced HyperPay, which is a multi-party protocol for the IoT settings. They use off-chain transactions by making a committee of users with verifiable random leader selection such that there's no central bottleneck. However, the HyperPay system relies on some blockchain infrastructure to settle disputes and doesn't use hash chains to issue payment tokens.

In [3], the authors introduced transferable payment channels for micropurchase transactions in their recent paper. In the work, they propose using smart contracts to allow secure and fair exchanges of small sums of money without the use of a broker. Their work is related because it focuses on micropayments, but each pair of users needs a separate on-chain payment channel.

In 2024, Sravan et al. [9] proposed LIO-PAY which supports off-line payments using LoRa-enabled devices, even in those places where there's no Internet access. Though they addressed several payment challenges specific to the IoT systems, double-spending is still possible from the perspective of decentralized validation without miners.

B. Payment Channel Networks

Payment Channel Networks (PCNs) are the dominant approach for scaling payments in blockchains. Benedetti et al. [4] analyzed potential PCN topologies for Central Bank Digital Currencies. The semi-hierarchical PCNs are proposed as more scalable compared to the decentralized ones because they give higher throughput and higher fault tolerance. In [10], Zhang and Qian introduced Aggregated PCNs (Agg-PCNs), in which only users hold a pool of assets and each asset could be potentially used in any channel of the network, whereas users rely on TEE (Trusted Execution Environment) to run honest computations to protect against honest behavior. Numerous papers address the security of PCNs: Weintraub et al. [5] analyze the Lightning network, detailing two of its vulnerabilities (payout race and channel congestion). Aumayr et al. [11] presented an attack called "timelock bribing" where malicious miners can use large funds in Lightning channels to steal timelocked funds from other parties. Furthermore, their construction doesn't need parties to stay online.

To prevent channel exhaustion, Flexichain, a system designed by Mohanty and Tripathy [2], utilizes flexible fund allocation within channels. Privacy is crucial in PCNs and researchers proposed different ways to achieve it. Panwar et al. [6] introduced SPRITE, a privacy-preserving routing protocol using two phases of transaction processing. In [7], LightPay, a multipath payment scheme based on adapter signatures, was proposed by Liu et al. for private transactions on the Lightning network. Podiatchev et al. [12] studied survivability in PCNs. In [13], a card payment protocol via PCN for cryptocurrency micro-payments was suggested by De Silva et al. Meanwhile, Xiao et al. [14] proposed a supervisable multiparty payment channel supporting both on-chain and off-chain payments.

The main caveat of all above approaches is that users need to pre-fund their channels and orchestrate routing by coordinating the state of all channel participants. This complicates the setup and operation significantly. Compared to that, the solution we proposed leverages hash-chain token flow and a fixed set of validators, leading to a simpler structure that's much better suited for resource-constrained IoT devices.

C. Double-Spend Prevention Methods in General

Double-spending is the key vulnerability of payment networks. In [8], Bazzi and Tucci-Piergiovanni analyze "fractional spending" where they show that payments are authenticated by receiving less than $f + 1$ replies if quorums exist and byzantine quorum intersection works without consensus. This work is related to ours in that it uses transaction authentication with less than a majority of nodes, however, the payment model is very general with balance-based payment with probabilistic quorum. It doesn't account for the semantics of hash-chain token payments, race conditions created by simultaneously multiple uses of a token, or locks and lease mechanisms designed to solve such conflicts. Our solution provides a guaranteed correct and secure behavior because the stability of our range through quorum intersection, with the locks to correctly serialize transactions reflecting the semantics of hash-chain tokens in the IoT ecosystem, provides protection. Hu et al. [15] proposed linkable ring signatures that can link multiple signatures created by a single user. Such an option is useful for tracing double-spending occurrences in an identity-centric digital payment system, but such signatures incur considerable computational costs and can be prohibitive for resource-limited IoT devices in terms of energy and compute requirements.

D. A Comparison to Our Research

Binary attributes of Table I reflect our methodological differences from related research. We further elaborate on the differences between our work and previously published papers below.

TABLE I: Comparison with Related Work

Work	Hash Chain	No Blockchain	Quorum	Lock/Lease	IoT Target
He et al. [1]	–	–	✓	–	✓
Payeras-C. [3]	–	–	–	–	✓
Bazzi et al. [8]	–	✓	✓	–	–
Sravan et al. [9]	–	✓	–	–	✓
Ours	✓	✓	✓	✓	✓

Our work integrates all features; hash chain, quorum based deterministic validity and lock/lease mechanisms, in a single protocol that's off-blockchain. While all these components have been featured separately in previous research, this particular combination is introduced to address the challenge of the race condition induced by a buyer sending the same hash-chain token to multiple merchants in small, permissioned, IoT networks.

III. Lock-Assisted Quorum Validation Model

The rest of the paper follows. We introduce the system model, hash-chain generation, the proposed validation protocol, and its safety and liveness guarantees.

A. System Model

The system involves three main roles:

- **Payer.** The payer is a single IoT device responsible for making micropayments. Once setup, the payer first creates a hash chain and shares the $chain\_id$ with validators. Validators then use $chain\_id$ to namespace their spent-token maps and lock-state maps. Validators do not know the chain anchor; instead, they verify token authenticity by reading the predecessor token supplied together with the current token in the payment request. The chain identity, denoted $chain\_id$, is a hash that uniquely identifies the payer. In practice, this means a chain identifier must be minted and transmitted to validators before the actual usage of hash-chain tokens. For this proof of concept, we assume that this setup takes place before any transaction.
- **Merchants ($M_1, M_2, \ldots$).** Merchants are vendors selling products or services. After receiving a token from the payer, a merchant contacts all validators and waits for at least q confirmations before approving the payment. The threat we focus on is a malicious payer attempting to create a race condition by submitting the same token to different merchants simultaneously.
- **Validators ($V_1, V_2, \ldots, V_n$).** There are n permissioned entities responsible for validating payments. All spent tokens and active locks are tracked by these n validator servers. The validator set $V_1, \ldots, V_n$ is static throughout the protocol and is known to all participants in advance. Dynamic validator membership is outside the scope of this work.

Validators exchange messages over a single shared network. We consider two distinct fault types: up to f_b Byzantine faults, where validators may behave arbitrarily, and up to f_c crash faults, where validators silently stop responding. These are analyzed separately: safety relies on protection against at most f_b Byzantine faults, while liveness depends on resilience against up to f_c crash faults. Merchants require at least q responses before approving a payment. We consider only one active payer and one hash chain. Extension to multiple concurrent payers is outside the scope of this work.

B. Threat Model and Assumptions

In this protocol, there are two kinds of faults: crash faults, in which a server crashes and stops responding and up to f_c such faults may occur, and Byzantine faults, in which a server can behave arbitrarily, including signing contradictory transactions, sending conflicting replies to different merchants, or colluding with a payer, and up to f_b such faults may occur. These two fault types are studied independently. The value of f_b affects the safety guarantee of the protocol, while f_c affects the liveness guarantee. We therefore analyze Byzantine faults and crash faults separately, setting $f_b{=}0$ for crash-fault tests and $f_c{=}0$ for Byzantine-fault tests. The payer may be malicious and may attempt to send the same token to two or more merchants at the same time to initiate multiple conflicting payment transactions. We exclude attacks in which the sender generates a fake chain, since that would require breaking the preimage resistance of the hash function.

Merchants. Merchants are assumed to be honest. They act correctly according to the protocol specification and do not collaborate with a malicious payer. Consequently, we do not need to defend against merchant-side misbehavior such as replaying old tokens or submitting fake messages.

Network model. We work with a partially synchronous distributed system. Messages are not guaranteed to arrive within fixed bounds or in order, but they are eventually delivered. There is no synchronized global clock. Instead, each server independently tracks the time elapsed for its own active lock using its private clock. We denote this time limit by Δ, and choose Δ to be longer than typical message-delivery times. In the rare case where a message takes longer than Δ to arrive, we sacrifice liveness to preserve safety: a server may release its lock due to timeout before the corresponding commit request arrives. This can lead to a failed legitimate payment, but it does not compromise double-spend safety. We assume there are no external attacks that stop servers from operating properly and no network partition that can isolate the majority of servers for a prolonged period.

C. Hash-Chain Token Generation

Let $H(\cdot)$ be a hash function (e.g. SHA-256). We define the sequence of hashes

$$t_0 = H(s), \quad t_i = H(t_{i-1}) \quad \text{for } i = 1, 2, \ldots, L-1 \tag{1}$$

The generated hash t_0 is the chain anchor. The chain of hashes is identified by $chain\_id$, which we generate by taking the first 16 hexadecimal digits of t_0 and transferring that identifier to validator nodes during setup. This gives each payment session related with a unique chain its unique identifier. The hash-chain tokens are consumed in reverse order, meaning that the k-th payment uses t_{L-k} paired with t_{L-k-1}.

Each server can check validity with single step. Once a server receives t_i along with t_{i-1}, it checks:

$$H(t_{i-1}) = t_i. \tag{2}$$

A server does not store the anchor and instead reads the previous token directly using message. This is secure because a party that lacks a valid predecessor token cannot construct an ordered hash pair, such as (a, a'), outside the valid hash chain that still satisfies $H(a) = a'$. An anchor must be pre-registered before token use, but that step is outside the capability of this proof of concept. We therefore assume that a setup phase with registered chain anchors has already taken place before the actual use of the hash-chain token.

If servers maintain a list of spent tokens, previously spent tokens can't be used. Thus, a server checks for the existence of the unique token $(chain\_id, i)$ in its spent list and denies the token request if such token ID already exists. The primary benefit is that each verification of a token is just one single hash operation which can still be performed in some low-power IoT devices.

D. Lock/Lease Validation Protocol

The validation process involves two phases, lock and commit, and is designed to prevent double-spending that could otherwise occur when the same token is submitted concurrently to multiple merchants.

Phase 1 – Lock Request: When a merchant receives token t_i, it sends a lock request message containing the tuple $(chain\_id,\ i,\ t_i,\ t_{i-1},\ merchant\_id)$ to the n validators. Each validator run three checks with received message:

1) Is the token valid? (Check $H(t_{i-1}) = t_i$ using t_{i-1} from the message.)
2) Is $(chain\_id, i)$ not already in the spent set?
3) Is $(chain\_id, i)$ not currently locked by another merchant?

If all checks succeed, the validator locks the token for that merchant for a lease duration Δ and replies with `LOCKED`. If the token is already spent or already locked, it replies with `REJECTED`. If the merchant sends no commit message within Δ seconds, the lock expires and the token becomes available to other merchants. If multiple merchants attempt to lock the same token simultaneously, each validator grants the lock to the first merchant whose request arrives and rejects later conflicting requests. Because validators do not share state during this phase, the same token can temporarily be locked by different merchants at different validators. However, due to quorum intersection (Section III-E), no single merchant can successfully obtain q confirmations in conflicting executions.

Phase 2 – Commit: If the merchant gets q or more lock confirmations, it sends a commit request to all validators that accepted its lock request. These validators check whether the token is still locked by the same merchant, move it to the spent set, and reply with `COMMITTED`.

If the merchant cannot obtain q lock confirmations, for example because another merchant captured the token first, the transaction fails. Locks time out automatically, so no token remains locked indefinitely.

E. Safety Condition

The safety property hinges on quorum intersection. When two merchants attempt to validate the same token, the quorums they collect must contain at least one common honest validator.

Let n be the number of validators, q the quorum size, and f_b the number of byzantine validators. It is a standard result that any two quorums of size q must have at least $2q-n$ validators in common, and we require this overlap to be greater than f_b so that it includes at least one honest validator:

$$2q - n \geq f_b + 1 \tag{3}$$

Argument. Whenever this condition holds, a merchant requesting validation has at least one honest validator in its quorum. Honest validators process lock requests sequentially: they grant the first lock request and reject later conflicting ones. As a result, the second merchant cannot collect q approvals, and double-spending is prevented. This argument holds under partial synchrony with up to f_b malicious validators, but it no longer holds if more than f_b byzantine validators collaborate.

As an example, for n=5 and f_b=2, we need $2q - 5 \geq 2 + 1 = 3$, which implies $q \geq 4$. If we choose q=4, then any two quorums of size 4 overlap in 3 validators. Even if 2 of those validators are byzantine, the remaining honest validator blocks the double-spend.

F. Liveness Condition

The system remains operational only if enough validators can still form a quorum. If at most f_c validators fail, then $n - f_c$ validators remain running, and we require this number to satisfy:

$$n - f_c \geq q \tag{4}$$

If each validator is online with probability p, then the probability that a merchant gets q or more replies can be computed using the binomial distribution:

$$P_{\text{live}}(n, q, p) = \sum_{i=q}^{n} \binom{n}{i} p^i (1-p)^{n-i} \tag{5}$$

This expression measures availability. For n=5, q=3, and p=0.9, the availability is 99.1% because $P_{\text{live}} = 0.991$.

G. Safety–Liveness Trade-off

The choice of quorum size q creates a safety–liveness trade-off. A larger q improves security by ensuring greater overlap, which helps detect more byzantine validators, but it reduces liveness because more validators must be online. The safety condition (3) and the liveness condition (4) establish the bounds on valid values of q:

$$\left\lceil \frac{n + f_b + 1}{2} \right\rceil \leq q \leq n - f_c \tag{6}$$

For this interval to be non-empty, we must have $\lceil (n+f_b+1)/2 \rceil \leq n - f_c$, or equivalently $f_b + 2f_c \leq n - 1$. When $f_b = f_c = f$, this reduces to the classical requirement $n \geq 3f + 1$. When n=5, f_b=1, and f_c=1, the only valid value is $q \in \{4\}$. However, if n=5, f_b=2, and f_c=2, then $q \geq \lceil (5+2+1)/2 \rceil = 4$ and $q \leq 5 - 2 = 3$, which means that no value $q \in \{0, 1, 2, 3, 4, 5\}$ is simultaneously safe and live. This is why f_b and f_c are tested separately. In practice, the system operator chooses a value of q within the valid range based on the priority assigned to safety versus availability.

IV. Analysis of the Proposed Solution

Through a controlled set of experiments on a prototype system, we provide experimental evidence for the validity of the lock/lease quorum validation method.

A. Experimental Setup

The prototype models each client and participant as an independent process. The payer, two merchants, and the validator quorum with n=5 validators run in separate Docker containers on a local network using HTTP. This provides process isolation and forces each validator to operate independently without access to the memory of other validators. Crash failures are simulated by terminating validator processes. Byzantine failures are simulated by configuring one or more validators to always approve all incoming lock requests. We run 100 trials for each experiment, except for the throughput test, which uses 50 runs, and all experiments use a 20-token hash chain.

We compare the lock/lease protocol with a baseline. In the baseline, a merchant sends a token to all validators. Upon receiving it, a validator checks whether it has already registered the token; if not, it marks the token as spent and replies with a success response. Unlike the lock/lease protocol, the baseline does not include a lock phase before updating token state. As a result, two merchants can send the same token at nearly the same time and both observe it as unspent before one validator records it, creating the race condition that the lock/lease protocol is designed to eliminate.

B. Double-Spend Prevention

We first evaluate the main security requirement of the system: preventing double-spends. In this experiment, the payer sends the same token to two different merchants at the same time, and both merchants attempt to validate it using the same validator set. We compare the baseline, which naively checks the spent state, with the proposed lock/lease protocol.

The results in Fig. 1 show that the lock/lease protocol prevents double-spends in all 100 tests, achieving 100/100 protection. As soon as a quorum of validators locks a token for one merchant, the other merchant cannot gather enough approvals to commit the same token. In contrast, the baseline fails to stop double-spends in 69 out of 100 trials. This failure occurs because checking whether a token is spent and marking it as spent are not atomic operations, so both merchants may obtain approval before the token is recorded as used. Unlike the baseline, whose results vary depending on scheduling, the lock/lease protocol removes this uncertainty and provides stable security behavior.

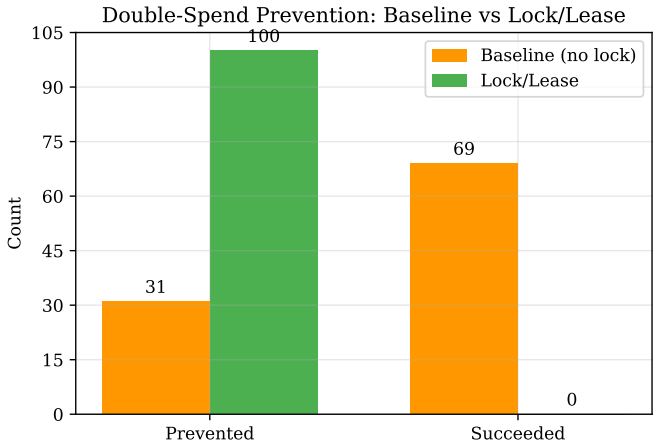


Fig. 1: Double-spend prevention comparison between baseline and lock/lease protocol (n=5, q=3, 100 runs).

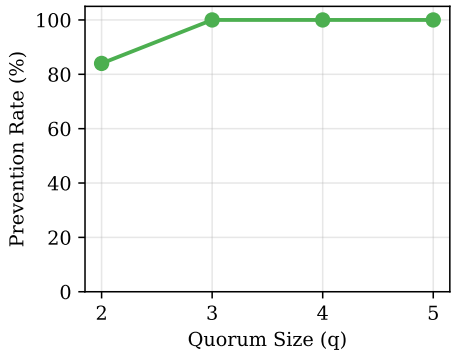
C. Effect of Quorum Size

Next, we investigate the impact of quorum size q on double-spend prevention using the lock/lease mechanism while setting n=5. We explore quorum sizes from 2 to 5.

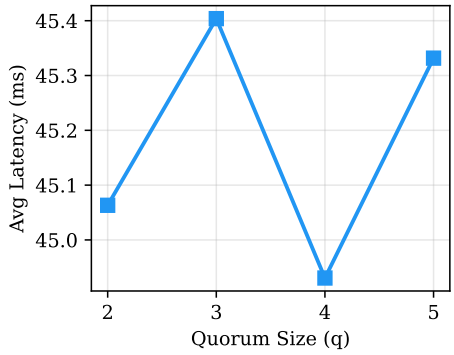
Figure 2 presents the results across three key metrics. Figure 2a indicates that when q=2, the double-spend prevention rate is 84%. This is because two quorums of size 2 may be completely disjoint, allowing conflicting requests to be approved independently. As the quorum size increases to 3 or more, the prevention rate rises to 100%. This follows from quorum intersection: when $2q > n$, any two quorums of size q are guaranteed to share at least one validator, and an honest validator in that overlap can detect a duplicate token. These findings apply to the fault-free setting (f_b=0, f_c=0). The implications of byzantine faults are discussed in Section IV-E.

The latency data is shown in Fig. 2b. The average latency remains between 44.9 and 45.4 ms for all tested values of q,

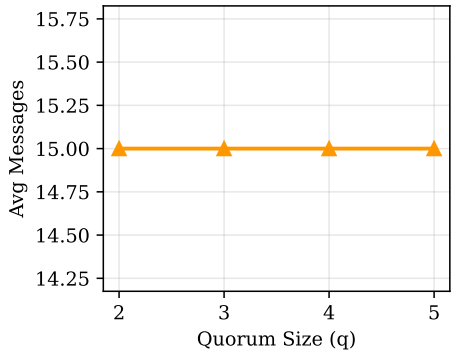
and the y-axis is intentionally compressed to make the small differences visible. Figure 2c shows that the communication cost remains constant at 15 messages per attempted double-spend. This is because each merchant sends one lock request and one commit request to each of the 5 validators, in addition to the initial payment exchange. Overall, increasing the quorum size improves security without significantly affecting latency or communication overhead.



(a) Double-spend prevention rate



(b) Average latency



(c) Constant communication overhead

Fig. 2: Effect of quorum size on system performance (n=5, lock/lease mode, 100 runs per q). Fig. 2b uses a narrow y-axis to show small latency differences, while Fig. 2c confirms a constant 15-message overhead.

D. Crash Fault Tolerance

We also assess crash fault tolerance for the lock/lease protocol using q=3 and n=5. In this experiment, we generate 100 payment requests at a constant rate and randomly crash f_c validators during execution.

Figure 3 shows the results. When f_c=2, there are still $n - f_c = 3$ active validators, which satisfies $n - f_c \geq q$. In this case, all 100 payments succeed and the success rate is 100%. When f_c=3, only 2 healthy validators remain, which violates $n - f_c \geq q$. The success rate therefore drops to 0%, and no payments complete successfully. These results match the liveness condition: the system remains operational only while the number of available validators is at least the quorum size. For the chosen parameters (n=5, q=3), the maximum tolerable number of crash faults is $f_{c,\max} = n - q = 2$.

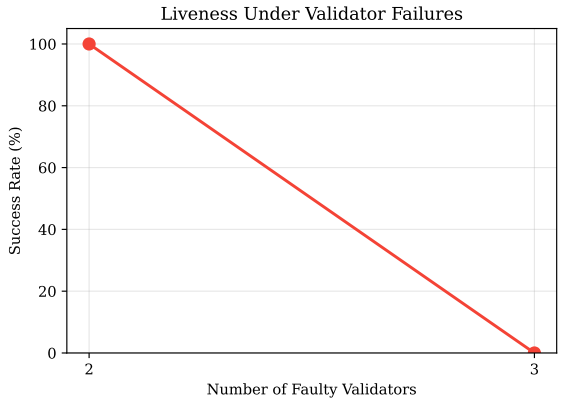


Fig. 3: Payment success rate under validator crash failures (n=5, q=3, lock/lease mode).

E. Safety Under Byzantine Faults

Byzantine validators approve conflicting double-spend transactions instead of rejecting them. Safety in this setting depends on the condition $2q - n \geq f_b + 1$, which guarantees that any two quorums intersect in at least one honest validator. To study this effect, we extend the quorum-size analysis from Section IV-C to configurations with byzantine faults, using f_c=0.

We perform double-spend experiments with $f_b \in \{1, 2\}$ and $q \in \{3, 4\}$. The results in Fig. 4 closely follow the theoretical bound. When q=4 and f_b=1, we have $2q - n = 2(4) - 5 = 3 \geq f_b + 1 = 2$, and the prevention rate is 100%. When q=4 and f_b=2, we again obtain full prevention because $2q - n = 3 \geq f_b + 1 = 3$. When q=3 and f_b=1, however, the condition fails because $2q - n = 2(3) - 5 = 1 < f_b + 1 = 2$, and the prevention rate drops to 97%. When q=3 and f_b=2, the safety condition is violated more strongly, with $2q - n = 1 < f_b + 1 = 3$, and the prevention rate falls to 84%.

These results are consistent with the theory. The lock/lease protocol remains fully safe against byzantine validators only when $2q - n \geq f_b + 1$ holds. If that condition is violated, some double-spend attempts succeed because the overlap

between competing quorums may consist entirely of byzantine validators that approve both requests. As f_b increases, the prevention rate drops further, which is exactly what the quorum-intersection analysis predicts.

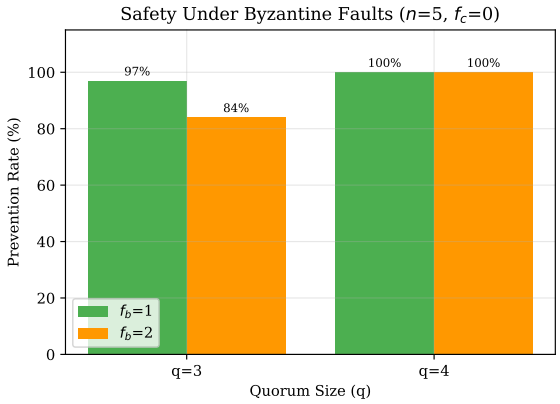


Fig. 4: Double-spend prevention rate under byzantine faults (n=5, f_c=0). The safety condition $2q-n \geq f_b+1$ determines whether full prevention is guaranteed.

F. Throughput and Latency

To evaluate practical performance, we feed a complete 20-token hash chain through the system sequentially. In this experiment, a single merchant redeems all 20 tokens in order, and we compare the lock/lease protocol with the baseline.

Figure 5 shows the results over 50 runs. Under the lock/lease protocol, the system achieves a throughput of 38.4 tokens/second with an average chain-processing time of 496.1 ms. The baseline reaches 32.1 tokens/second with an average chain-processing time of 592.0 ms.

This corresponds to a per-token latency of 24.8 ms in the lock/lease protocol ($496.1/20$), i.e., about 25 ms. Such latency is acceptable for IoT micropayment scenarios that require responses well below 1 second.

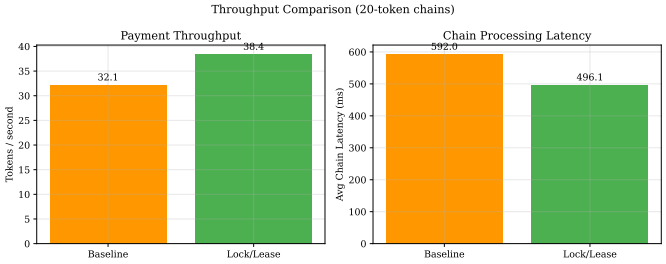


Fig. 5: Throughput and chain processing latency comparison between baseline and lock/lease (20-token chains, 50 runs). The baseline includes a deliberate race-window delay, so the figure should not be interpreted as showing an inherent speed advantage of locking.

G. Summary of Results

Table II summarizes the all findngs from all experiment.

TABLE II: Summary of Experiment Results

Experiment	q	**Faults**	**Prev. (%)**	**Lat. (ms)**
Double-Spend Prev. (f_b=f_c=0)				
Lock/Lease	3	—	100	45.4
Baseline	3	—	31	52.9
Quorum Sweep (f_b=f_c=0)				
Lock/Lease	2	—	84	45.1
Lock/Lease	3	—	100	45.4
Lock/Lease	4	—	100	44.9
Lock/Lease	5	—	100	45.3
Crash Tolerance (q=3, f_b=0)				
Normal pay	3	f_c=2 cr.	100†	41.4
Normal pay	3	f_c=3 cr.	0†	40.3
Byzantine Safety (f_c=0)				
Lock/Lease	3	f_b=1 byz.	97	45.9
Lock/Lease	3	f_b=2 byz.	84	47.7
Lock/Lease	4	f_b=1 byz.	100	45.0
Lock/Lease	4	f_b=2 byz.	100	46.5
Throughput				
			tok/s	**Chain (ms)**
Lock/Lease	3	—	38.4	496.1
Baseline	3	—	32.1	592.0

†Payment success rate (not prevention rate).

The three main results of the study are confirmed by the experiments. First, in tested configurations satisfying $2q-n \geq f_b+1$, we did not observe any double-spend in 100 runs. Second, when the condition $2q-n \geq f_b+1$ is violated, unsafe byzantine configurations allow double-spends to succeed. Third, liveness is preserved only when $n-f_c \geq q$; otherwise, the system cannot gather enough approvals to finalize payments. The prototype achieved 38.4 tokens/second throughput with about 25 ms per-token latency, which may be suitable for some small-scale IoT payment settings.

H. Discussion

The experimental results were consistent with the predictions made in the previous section, showing that the lock/lease mechanism behaves as expected.

Lease timeout and asynchrony. Because leases have a timeout duration, another participant may acquire the same lock if payment confirmation is delayed for too long. This does not violate safety, but it can reduce liveness because a valid transaction may fail. In a fully asynchronous network, message delays have no known upper bound, so this creates a direct tension between safety and liveness. Shorter leases are preferable in low-latency networks or when responsiveness is prioritized, whereas longer leases provide a larger safety margin in higher-latency environments.

Correlated validator failures. In the simplified availability model of (5), validator failures are assumed to be independent with probability p. In practice, however, several validators may fail together if they share the same provider, rack,

or data center. These correlated failures would change the observed availability. A fair deployment would place validators in independent failure zones. The more detailed availability analysis for correlated failures is left to future work.

We ran the baseline with a non-atomic `check-then-mark` operation, making it similar to a lockless system. Therefore, the throughput difference between the baseline ($\approx$ 32.1 tok/s) and the lock/lease protocol ($\approx$ 38.4 tok/s) is influenced mainly by the deliberate delay inserted to expose the race condition. The principal advantage of the lock/lease model is thus correctness and safety rather than raw performance.

Our experiments were conducted on local servers with near-zero communication latency. In realistic IoT environments, larger wireless or wide-area delays could increase lock contention and reduce throughput. Finally, the system assumes a fixed validator set known before operation. Any dynamic change in validator membership requires reconfiguration, which is a limitation for more flexible deployments.

V. Conclusion and Future Work

This paper showed that hash-chain token generation combined with a lock-assisted two-phase validation protocol can deter double-spending in hash-chain-based IoT micropayments. The system uses on a small set of permissioned validators and quorum-based approval. We analyzed safety using the condition $2q-n \geq f_b+1$ and liveness using the condition $n - f_c \geq q$ under a partially synchronous network model. The experimental results showed zero instances of double-spending in safe tested configurations, while the non-locking baseline failed in 69 out of 100 runs because of scheduling-related races. With n=5 and q=3, the system tolerates up to 2 crash failures, remains safe when the safety criterion is satisfied, and achieves up to 38.4 tokens/second with about 25 ms per-token latency, making it promising for small-scale IoT micropayments.

We presented a proof-of-concept lightweight lock-assisted protocol for small-scale deployment with static participants. This design introduces limitations, such as static membership and the lack of support for multiple simultaneous payers, but it allows the model to provide clear safety, liveness, and performance guarantees. The protocol also intentionally avoids advanced cryptographic primitives such as privacy-preserving signatures in favor of lightweight validation among known participants.

Future work will continue in three main directions. First, we will study scalability with larger validator sets such as n=7 and n=9. Second, we will evaluate the protocol under more realistic IoT conditions, including intermittent delays and packet loss. Third, we will investigate mechanisms for dynamic validator membership so that the system no longer requires manual reconfiguration during operation.

References

[1] J. He, W. Qiu, S. Zhuo, M. Xu, Q. Zhang, Z. Xiong, and Z. Zheng, "An efficient multiparty payment protocol for iot micro-payments," *IEEE Internet of Things Journal*, vol. 11, no. 20, pp. 32 854–32 867, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2024.3421245

[2] S. K. Mohanty and S. Tripathy, "Flexichain: Flexible payment channel network to defend against channel exhaustion attack," *ACM Transactions on Privacy and Security*, vol. 27, no. 4, pp. 30:1–30:26, 2024. [Online]. Available: https://doi.org/10.1145/3687476

[3] M. M. Payeras-Capellà, M. M. Puigserver, and R. Pericàs-Gornals, "Transferable channels for fair micropurchases," *International Journal of Information Security*, vol. 24, no. 1, p. 34, 2025. [Online]. Available: https://doi.org/10.1007/s10207-024-00929-6

[4] M. Benedetti, F. D. Sclavis, M. Favorito, G. Galano, S. Giammusso, A. Muci, and M. Nardelli, "An analysis of pervasive payment channel networks for central bank digital currencies," *Computer Communications*, vol. 240, p. 108199, 2025. [Online]. Available: https://doi.org/10.1016/j.comcom.2025.108199

[5] B. Weintraub, S. P. Kumble, C. Nita-Rotaru, and S. Roos, "Payout races and congested channels: A formal analysis of security in the lightning network," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, 2024, pp. 2562–2576. [Online]. Available: https://doi.org/10.1145/3658644.3670315

[6] G. Panwar, R. Vishwanathan, G. Torres, and S. Misra, "Sprite: Secure and private routing in payment channel networks," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIA CCS '24)*, 2024, pp. 1878–1894. [Online]. Available: https://doi.org/10.1145/3634737.3644995

[7] Y. Liu, W. Liang, K. Xie, S. Xie, K. Li, and W. Meng, "Lightpay: A lightweight and secure off-chain multi-path payment scheme based on adapter signatures," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1622–1635, 2024. [Online]. Available: https://doi.org/10.1109/TSC.2023.3333806

[8] R. A. Bazzi and S. Tucci-Piergiovanni, "The fractional spending problem: Executing payment transactions in parallel with less than f+1 validations," in *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing (PODC '24)*, 2024, pp. 295–305. [Online]. Available: https://doi.org/10.1145/3662158.3662817

[9] S. S. Sravan, S. Mandal, and P. J. A. Alphonse, "Lio-pay: Sustainable low-cost offline payment solution," *Electronic Commerce Research and Applications*, vol. 67, p. 101440, September–October 2024. [Online]. Available: https://doi.org/10.1016/j.elerap.2024.101440

[10] X. Zhang and C. Qian, "Toward aggregated payment channel networks," *IEEE/ACM Transactions on Networking*, vol. 32, no. 5, pp. 4333–4348, 2024. [Online]. Available: https://doi.org/10.1109/TNET.2024.3423000

[11] L. Aumayr, Z. Avarikioti, M. Maffei, and S. Mazumdar, "Securing lightning channels against rational miners," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, 2024, pp. 393–407. [Online]. Available: https://doi.org/10.1145/3658644.3670373

[12] Y. Podiatchev, A. Orda, and O. Rottenstreich, "Survivable payment channel networks," *IEEE Transactions on Network and Service Management*, vol. 21, no. 6, pp. 6218–6232, 2024. [Online]. Available: https://doi.org/10.1109/TNSM.2024.3456229

[13] A. D. Silva, S. Thakur, and J. G. Breslin, "Card payment protocol for cryptocurrencies with payment channel network," *Distributed Ledger Technologies: Research and Practice*, vol. 3, no. 3, pp. 19:1–19:30, 2024. [Online]. Available: https://doi.org/10.1145/3653681

[14] K. Xiao, J. Li, Y. He, X. Wang, and C. Wang, "A secure multi-party payment channel on-chain and off-chain supervisable scheme," *Future Generation Computer Systems*, vol. 154, pp. 330–343, 2024. [Online]. Available: https://doi.org/10.1016/j.future.2024.01.012

[15] M. Hu, Z. Liu, X. Ren, and Y. Zhou, "Linkable ring signature scheme with stronger security guarantees," *Information Sciences*, vol. 680, p. 121164, 2024. [Online]. Available: https://doi.org/10.1016/j.ins.2024.121164